Assignment 3 - Part 1 (Programming)

ECE/CS 5544 Spring 2026

Assigned: Thursday, March 26, 2026

Due: TBA

1 Abstract

In this assignment, you will write passes to improve code by eliminating redundant or unused computations. You are required to write three passes: Dominators, Dead Code Elimination (DCE), and Loop Invariant Code Motion (LICM). You will also be measuring the speedup of programs which have been optimized by your passes.

2 Policy

You are encouraged to work in groups of two to solve the problems for this assignment.

3 Submission

Both group members should upload the solutions. Please include the following items in an archive labeled with your pid (e.g., pid.tar.gz), and submit the resulting file to Canvas. With your source code, please make sure that you submit the following files:

```
./pid/unifiedpass.cpp
./pid/Makefile
./pid/tests/DCE
./pid/tests/Dominators
./pid/tests/LICM
./pid/README
./pid/writeup.pdf
```

- A report that briefly describes the implementations and results of all the passes, named `writeup.pdf`, containing both the group members' pid.
- Well-commented source code for your passes and associated Makefile.
- A `README` file describing how to build and run your passes.
- Any tests used for verification of your code.

4 Profiling with LLVM

You are required to measure the performance – in terms of *dynamic instruction count* – of the transformed bitcode generated by applying your LICM and DCE passes. You will measure the resulting program speedup.

Programs can be profiled in multiple ways. You could simply time your program over some number of iterations, but your results would be highly dependent on your particular machine's hardware and software configuration; however, this requires no changes to be made to the program under inspection.

Another way to estimate the performance of a program is to simply measure how many LLVM instructions are dynamically executed when it runs. To do this, you can use `lli`, the LLVM interpreter. Ordinarily, the interpreter will try to JIT-compile the bitcode passed to it, but you can force it to take the slow path (while counting instructions) by using the -force-interpreter option. You should always get the same instruction count every time you run `lli`. This approach works best with test programs that have a `main()` function. This is, of course, not a very good machine model; for example, all instructions are assigned the same cost (even pseudo-instructions, like `getelementptr`) and there is no notion of memory latency. As a first pass, though, it provides a nice way to measure the effectiveness of your passes.

In LLVM 14 and later, `lli` no longer prints the dynamic instruction count by default (although it continues to collect these statistics). Therefore, you must enable the printing of statistics. Depending on your environment, proceed as follows:

1. **If you are using the Docker image from the previous assignment:**
   - Pull the patched Docker image for this assignment:

     ```
     docker pull priyatam19/compiler-opts-lli-stats:llvm21
     ```

   - You may remove the old Docker image if you no longer need it; this assignment only requires the patched image.
   - Start the container as follows:

     ```
     docker run --rm -it -v "$PWD":/work -w /work \\
     priyatam19/compiler-opts-lli-stats:llvm21 /bin/bash
     ```

   - Inside the container, run:

     ```
     lli -stats -force-interpreter testcode.bc
     ```

2. **If you are using an LLVM source build:**
   - Download the provided patch file from Canvas.
   - Apply the patch to your LLVM source tree. Make sure the patch applies cleanly; if it does not, verify the changes manually and edit the file as needed.
   - Rebuild the updated `lli` tool from your build directory using Ninja or your normal CMake-based build flow. For example:

     ```
     cd <llvm-source-root>
     git apply print_dynamic_instr_count.patch
     cd <llvm-build-dir>
     ninja lli
     ```

   - You can then run:

     ```
     lli -stats -force-interpreter testcode.bc
     ```

You should then see the dynamic instruction count printed as follows:

5 Analysis Passes

5.1 Dominators (Purple Dragon Book 9.6.1)

You will first need to implement your own pass to calculate dominance information over loops. Please call your pass `dominators`. It is recommended to derive from LLVM's FunctionPass class. For each loop you

```
===-------------------------------------------------------------------------===
                          ... Statistics Collected ...
===-------------------------------------------------------------------------===

39 bitcode-reader - Number of Metadata records loaded
 2 bitcode-reader - Number of MDStrings loaded
13 interpreter    - Number of dynamic instructions executed
```

Figure 1: Printing the number of dynamic instructions executed

process, print out the names and immediate `dominators` of the blocks belonging to the loop (to standard output; names should come from `BasicBlock::getName()`). You should implement your own dataflow analysis framework to implement this pass (instead of using LLVM's existing pass for computing dominators).

5.2 Dead Code Elimination (Purple Dragon Book 9.1.6)

In this pass, you will remove unused (dead) instructions and preserve live ones, using the faint analysis from assignment 1. Compute the set of instructions to remove or preserve; then use it to eliminate instructions appropriately. Use your dataflow analysis framework to process sets of instructions. Please call your pass `dead-code-elimination`.

You may study the code listing in Figure 2 to get help on determining live variables.

```
bool isLive(Instruction *I) {
    return isa<TerminatorInst>(I) || isa<DbgInfoIntrinsic>(I) ||
           isa<LandingPadInst>(I) || I->mayHaveSideEffects();
}
```

Figure 2: Some conditions for determining instruction liveness.

For faint analysis, you may use your own analysis from assignment 1, the lecture notes, or you may use the algorithm listed in Figure 3 of the paper: "Complexity of Data Flow Analysis for Non-Separable Frameworks", B. Karkare and U. Khedker, arXiv, October 2006, which is also shown in Figure 3.

- Lhs_n : Singleton set containing LHS variable of statement n.
- Rhs_n : Set of variables used in RHS expression of statement n which is an assignment statement. For non-assignment statements like *print* and for function calls, this set is empty.
- Use_n : Set of variables used in statement n, where n is either a non-assignment statement or a function call.
- $\mathsf{FaintGen}_n/\mathsf{FaintKill}_N$: Set of variables whose faintness is generated/killed by statement n.
- $\mathsf{FaintIn}_n/\mathsf{FaintOut}_n$: Faintness information at entry/exit of node n.

$$
\begin{aligned}
\mathsf{FaintGen}_n &= \{x \mid x \in \mathsf{Lhs}_n, x \notin \mathsf{Rhs}_n\} \\
\mathsf{FaintKill}_n &= \{x \mid x \in \mathsf{Rhs}_n, y \in \mathsf{Lhs}_n, y \notin \mathsf{FaintOut}_n\} \\
&\quad \cup \{x \mid x \in \mathsf{Use}_n\} \\
\mathsf{FaintIn}_n &= (\mathsf{FaintOut}_n - \mathsf{FaintKill}_n) \cup \mathsf{FaintGen}_n \\
\mathsf{FaintOut}_n &= \begin{cases} \top & n \text{ is exit node} \\ \bigcap_{s \in succ(n)} \mathsf{FaintIn}_s & \text{otherwise} \end{cases}
\end{aligned}
$$

Figure 3: Data flow equations to determine faint variables.

Include at least two microbenchmarks that you wrote in your submission. In your writeup, discuss the changes in dynamic instruction count on the transformed bitcode after running your pass on each microbenchmark.

5.3 Loop Invariant Code Motion (Purple Dragon Book 9.5.1)

In this pass, you will decrease the number of instructions executed during a loop by identifying and hoisting out those that are loop-invariant, as discussed in the lectures from class. In addition to hoisting out candidate instructions that dominate all exits of the loop, you will need to implement the landing pad transformation discussed in the lecture to handle loops with a zero iteration path through the loop. This will allow candidate statements that do not dominate all exits to be hoisted into the new preheader. Please call your pass `loop-invariant-code-motion`.

For more details on the landing pad transformation, please see Section 4.2 of the paper: "Global Value Numbers and Redundant Computations", B. K. Rosen, M. N. Wegman, and F. K. Zadeck, ACM, 1988.

It is recommended to derive from LLVM's LoopPass class. You should use dominator information provided by your pass, and loop information provided by the built-in `LoopInfo` pass, except for methods related to loop-invariance (`isLoopInvariant()`, `hasLoopInvariantOperands()`, `makeLoopInvariant()`, etc). Refer to the LLVM documentation for specifying interactions between passes.

You may assume that the input received by your pass has already been transformed by the `loop-simplify` pass to insert loop preheaders where appropriate. If this built-in pass is unable to insert a preheader (e.g., `((Loop*) foo)->getLoopPreheader() == NULL)`, you may ignore the loop.

For each loop, compute the set of loop-invariant instructions. You may ignore child nested loops that you have already processed, but you should ensure that deeply-nested loop-invariant computations can still bubble all the way out. Use the guidelines from the class notes. Hoist to the preheader all loop-invariant

instructions that are candidates for code motion, ensuring that dependencies are preserved.

You may study the code listing in Figure 4 to get help on determining loop invariant instructions.

```
#include "llvm/Analysis/ValueTracking.h"

bool isInvariant(Instruction *I) {
    return isSafeToSpeculativelyExecute(I) && !I->mayReadFromMemory() &&
           !isa<LandingPadInst>(I);
}
```

Figure 4: Some conditions for determining invariant instructions.

Include at least three microbenchmarks that you wrote in your submission. In your writeup, discuss the changes in dynamic instruction count on the transformed bitcode after running your pass on each microbenchmark.

To illustrate LICM, see the example code in Figure 6 in Appendix 7. This code is lowered into LLVM bitcode in Figure 7 and LICM-transformed in Figure 8. Also, you may refer to Section 7.3 in Appendix 7 which clarifies SSA format with respect to LICM.

6 Reference

This assignment is inspired by CMU 15-745 course's assignment and is reused here with permission. The original assignment can be found here.

7 Appendix: Test cases

This section provides reference test cases and important implementation notes for the Dominators, Dead Code Elimination (DCE), and Loop Invariant Code Motion (LICM) passes.

7.1 Important Note

These test cases are for your reference only. You must create and include your own *custom test cases* to receive credit for this section. You are encouraged to demonstrate correctness, coverage, and optimizations made by your passes.

Dominators:

Test case:

```c
int main ()
{
  int result;
  int n = 1;

  while (n++ < 100) {
          result = 0;
          result += 2;
          result += 3;
          result *= 4;
          result /= 2;
  }

  return result;
}
```

Expected Result:

```
while.cond is dominated by entry
while.body is dominated by while.cond
while.end is dominated by while.cond
```

DCE:

```c
int main(int argc, const char * argv[]) {
    int x = 1;
    int a = x + 50;
    int b = a + 96;
    int c;
    int d;
    if(a > 50) {
        c = a - 50;
        d = a * 96;
    } else {
        c = a + 50;
        d = a * 96;
    }
    int e = 50 -96;
    int f = e + c;
    return 0;
}
```

Expected result:

```
user@user-VirtualBox:~/Assignment3/nitheeshp/DCE$ opt -enable-new-pm=0 -load ./
Instructions that can be deleted
  %add1 = add nsw i32 %add, 96,
  %sub = sub nsw i32 %add, 50,
  %mul = mul nsw i32 %add, 96,
  %add2 = add nsw i32 %add, 50,
  %mul3 = mul nsw i32 %add, 96,
  %c.0 = phi i32 [ %sub, %if.then ], [ %add2, %if.else ],
  %add4 = add nsw i32 -46, %c.0,
Removing Instruction   %add4 = add nsw i32 -46, %c.0
Removing Instruction   %c.0 = phi i32 [ %sub, %if.then ], [ %add2, %if.else ]
Removing Instruction   %mul3 = mul nsw i32 %add, 96
Removing Instruction   %add2 = add nsw i32 %add, 50
Removing Instruction   %mul = mul nsw i32 %add, 96
Removing Instruction   %sub = sub nsw i32 %add, 50
Removing Instruction   %add1 = add nsw i32 %add, 96
```

Figure 5: Test Case: Dead Code Elimination (DCE)

7.2 LICM: Illustration

```c
#include <stdio.h>

int test(int x){
  int p = 0;
  for (int i = 0;i < 100; i++){
    p = x*2;
  }
  return p;
 }

int main(){
  int r = test(3);
  printf("result is: %d\n",r);
  return 0;
}
```

Figure 6: Example input code for illustrating LICM.

```
@.str = private unnamed_addr constant [15 x i8] c"result is:
%d\0A\00", align 1

; Function Attrs: noinline nounwind uwtable
define i32 @test(i32) #0 {
  br label %2

; <label>:2:                                      ; preds = %6,
%1
  %.01 = phi i32 [ 0, %1 ], [ %5, %6 ]
  %.0 = phi i32 [ 0, %1 ], [ %7, %6 ]
  %3 = icmp slt i32 %.0, 100
  br i1 %3, label %4, label %8

; <label>:4:                                      ; preds = %2
  %5 = mul nsw i32 %0, 2
  br label %6

; <label>:6:                                      ; preds = %4
  %7 = add nsw i32 %.0, 1
  br label %2

; <label>:8:                                      ; preds = %2
  ret i32 %.01
}

; Function Attrs: noinline nounwind uwtable
define i32 @main() #0 {
  %1 = call i32 @test(i32 3)
  %2 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds
([15 x i8], [15 x i8]* @.str, i32 0, i32 0), i32 %1)
  ret i32 0
}
declare i32 @printf(i8*, ...) #1
```

Figure 7: LLVM bitcode of example in Figure 6.

```
@.str = private unnamed_addr constant [15 x i8] c"result is:
%d\0A\00", align 1

; Function Attrs: noinline nounwind uwtable
define i32 @test(i32) #0 {
  %2 = mul nsw i32 %0, 2
  br label %3

; <label>:3:                                        ; preds = %6,
%1
  %.01 = phi i32 [ 0, %1 ], [ %2, %6 ]
  %.0 = phi i32 [ 0, %1 ], [ %7, %6 ]
  %4 = icmp slt i32 %.0, 100
  br i1 %4, label %5, label %8

; <label>:5:                                        ; preds = %3
  br label %6

; <label>:6:                                        ; preds = %5
  %7 = add nsw i32 %.0, 1
  br label %3

; <label>:8:                                        ; preds = %3
  ret i32 %.01
}

; Function Attrs: noinline nounwind uwtable
define i32 @main() #0 {
  %1 = call i32 @test(i32 3)
  %2 = call i32 (i8*, ...) @printf(i8* getelementptr inbounds
([15 x i8], [15 x i8]* @.str, i32 0, i32 0), i32 %1)
  ret i32 0
}

declare i32 @printf(i8*, ...) #1
```

Figure 8: LICM-transformed bitcode of example in Figure 6.

7.3 The Impact of SSA format on LICM

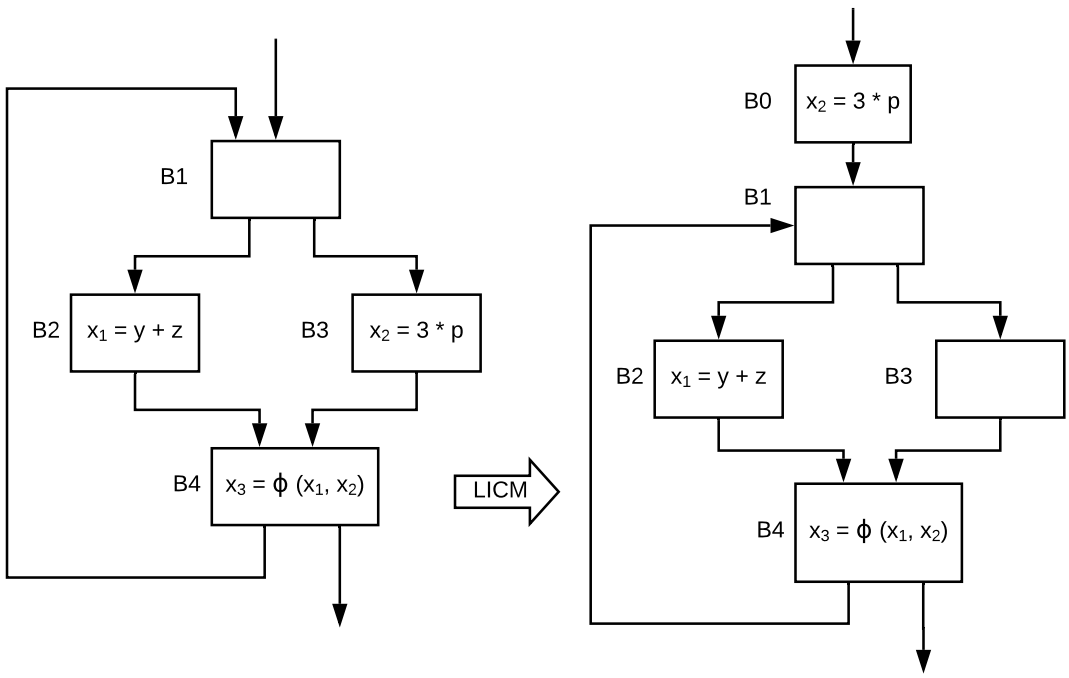

Figure 9: The impact of SSA format on LICM.

In this assignment, we do not assume that the input LLVM bitcode is in non-SSA format. SSA format still helps with LICM. Suppose, in a loop, for a phi node, $x_3 = \phi(x_1, x_2)$ and $x_2 = 3 * p$, where p is another loop-invariant variable. We can then move $x_2 = 3 * p$ into the preheader and thereby avoid computing the same $3 * p$ expression each time we take the branch that computes $3 * p$. Figure 9 illustrates this scenario.

7.4 Landing Pad Insertion Notes

Landing pad insertion is a crucial component in our implementation of Loop Invariant Code Motion (LICM). Its purpose is to allow the safe hoisting of loop invariants that do not dominate every exit of the loop, by ensuring that such invariants are only executed when the loop actually runs (i.e., when the initial test condition is satisfied). Note that you **must not** use the existing LLVM pass `loop-rotate` for this task. Instead, follow the guidelines provided below.

Guidelines for Landing Pad Insertion:

- **Basic Block Insertion:** Insert two new basic blocks between the original preheader and the loop header:

  1. A *conditional test block* that replicates the branch condition of the original loop header.
  2. An *unconditional landing pad block* where the loop invariants (candidates for hoisting) are placed.

  The conditional test block should also provide an edge to the loop exit, ensuring that the landing pad is only reached if the loop is executed at least once.

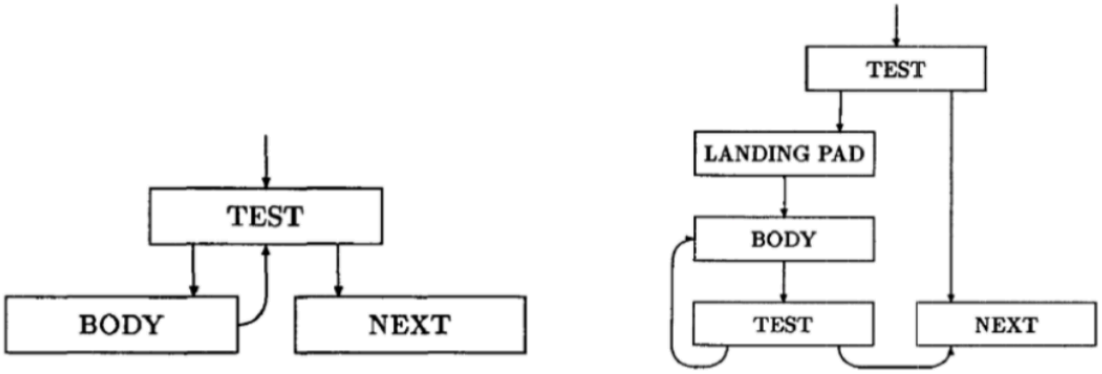

Figure 10: Landing Pad during Loop Invariant Code motion

- **Instruction Manipulation:** Several helper functions can simplify your implementation:
  - `Instruction::clone()` to create a duplicate of an instruction.
  - `Instruction::op_iterator` for iterating over instruction operands.
  - `setSuccessor(unsigned, BasicBlock *)` to update the control-flow graph (CFG) edges.
  - `llvm::ReplaceInstWithInst(Instruction *From, Instruction *To)` to safely replace instructions.
  - `insertBefore(Instruction *InsertPos)` to insert new instructions in a basic block.
- **Dominance and Reaching Definitions:** Even in SSA form, not all definitions necessarily dominate every loop exit. Therefore, you must use information from a Dominators Pass and a Reaching Definitions analysis to determine which invariants can be safely hoisted to the preheader via the landing pad. For example, if the loop body contains an exit edge (such as from a `break` statement) that bypasses some definitions, blindly hoisting all invariants could lead to unsafe transformations.
- **Conservative Code Motion:** The minimum requirement is to implement LICM and the landing pad transformation conservatively. That is, only hoist those invariants that are guaranteed to be safe based on the dominator and reaching definition analyses. While more aggressive optimizations (e.g., incorporating liveness or exception analysis) are possible as extra credit, your base implementation must adhere to the guidelines described here.

Discussion: If one were to move all loop invariants to the preheader without considering dominance relationships, the following issues may arise:

- In cases where loop invariants do not dominate all loop exits, moving them unconditionally can lead to incorrect behavior. For instance, if a loop body contains an exit edge (e.g., due to a `break` statement), some invariants might not be executed when the loop exits early.
- The safe hoisting of invariants relies on the fact that only the statements in the *test block* (created by the landing pad transformation) dominate the loop exit. Therefore, additional analysis using dominators and reaching definitions is required to ensure that only the correct invariants are hoisted.

Extra credit. We encourage more aggressive optimizations by relaxing the code motion rules as much as possible. This would be extra credit and should be clearly documented in your report.